

Elaborazione di Segnali Multimediali

Image Restoration

D. Cozzolino, L. Verdoliva

In questa esercitazione useremo le reti neurali convoluzionali per realizzare image restoration. In questa esercitazione useremo il Berkeley Segmentation Data Set (<https://github.com/BIDS/BSDS500>), formato da 500 immagini a colori. Per usarlo, abbiamo bisogno del modulo Python `BSDS500.py`, che può essere scaricato in Colab con la seguente istruzione:

```
!wget -c -q https://www.grip.unina.it/download/guide_TF/BSDS500.py
```

Prima di iniziare, resettiamo la console e importiamo le librerie che useremo:

```
%reset -f
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.utils.data as data
import tqdm
from torchvision import transforms
```

1 Denoising

Per rimuovere rumore AWGN, utilizzeremo l'architettura DnCNN che è una rete fully convolutional in quanto non prevede strati *fully connected*. Il vantaggio delle reti fully convolutional è di poter essere applicate a immagini di diverse dimensioni. L'architettura DnCNN che useremo è mostrata in figura 1. Per comodità dividiamo il codice in quattro parti: definizione dell'architettura, preparazione dei dati, addestramento e analisi delle prestazioni.

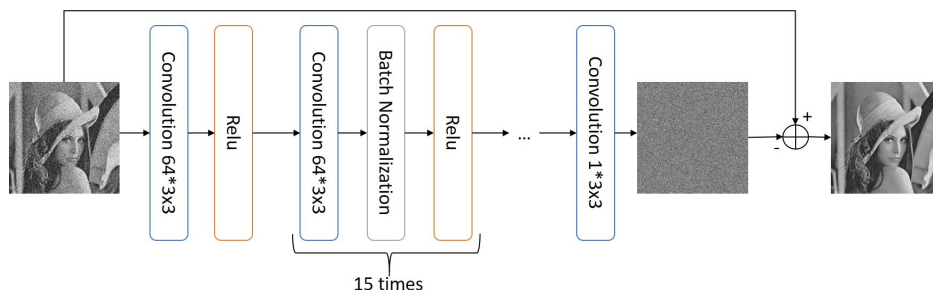


Figura 1: Architettura DnCNN

1.1 Definizione dell'architettura

L'architettura DnCNN, oltre a non prevedere strati *fully connected*, ha due caratteristiche rilevanti. La prima è l'uso di strati di *Batch Normalization*, che rendono l'addestramento più veloce e stabile. L'altra caratteristica è l'impiego del *Residual Learning*: la rete non predice direttamente l'immagine ripulita, ma il residuo di rumore, che viene sottratto all'immagine di ingresso per ottenere l'immagine filtrata. Per comodità, definiamo la rete tramite una funzione:

```
def DnCNN(img_channels=3, num_features=64, depth=17):
    layers = [
        nn.Conv2d(img_channels, num_features, kernel_size=3, padding=1, bias=True),
        nn.ReLU(),
    ]
    for index in range(depth - 2):
        layers += [
            nn.Conv2d(num_features, num_features, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(num_features),
            nn.ReLU(),
        ]
    layers += [
        nn.Conv2d(num_features, img_channels, kernel_size=3, padding=1, bias=True),
    ]
    model = nn.Sequential(*layers)
    for m in model.modules():
        if isinstance(m, nn.Conv2d):
            nn.init.orthogonal_(m.weight) # initialize weights
            if m.bias is not None:
                nn.init.zeros_(m.bias) # initialize biases
    return model

device = "cuda:0" if torch.cuda.is_available() else "cpu"
model = DnCNN(img_channels=3).to(device)
```

1.2 Preparazione dei dati

Prima di definire gli oggetti dataset occorre definire le operazioni di preprocessing, indicando anche la data augmentation per il training. Inoltre, per questioni di memoria la rete verrà addestrata utilizzando blocchi estratti random di dimensioni 64×64 pixel.

```
transform_train = transforms.Compose([
    transforms.RandomCrop((96, 96)),
    transforms.RandomAffine(degrees=10, translate=(0.05, 0.05), scale=(0.8, 1.2)),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.05),
    transforms.CenterCrop((64, 64)),
    transforms.ToTensor(),
])
transform_eval = transforms.ToTensor() # dimensioni effettive delle immagini in eval
```

A questo punto potete eseguire le seguenti istruzioni per creare gli oggetti dataset per i tre set:

```
from BSDS500 import BSDS500
train_set = BSDS500("./data", split="train400", download=True, transform=transform_train)
val_set = BSDS500("./data", split="val32", download=True, transform=transform_eval)
test_set = BSDS500("./data", split="test68", download=True, transform=transform_eval)
train_set = data.ConcatDataset(32*[train_set,]) # 32 blocchi da ogni immagine per epoca
print(len(train_set), len(val_set), len(test_set))
```

Notate che il dataset di training è stato esteso di 32 volte, in questo modo per ogni epoca sono esaminati 12800 blocchi in totale (32 blocchi per ciascuna delle 400 immagini di training). Creiamo i `DataLoader` per ciascun insieme, specificando il `batch_size`:

```
batch_size = 200
train_loader = data.DataLoader(train_set, batch_size=batch_size, shuffle=True)
val_loader = data.DataLoader(val_set, batch_size=1, shuffle=False)
test_loader = data.DataLoader(test_set, batch_size=1, shuffle=False)
```

Per la validation e il test set usiamo un batch size pari a 1, perché le immagini hanno dimensioni diverse e, pertanto, non possono essere unite in batch. Infine definiamo una *degradation function* per aggiungere rumore gaussiano alle immagini con una deviazione standard fissa pari a 50:

```
def add_noise(images, sigma = 50):
    # images in range [0, 1]
    # sigma relative to range [0, 255]
    noise = (sigma / 255.0) * torch.randn_like(images)
    return images + noise
```

1.3 Addestramento

Per l'addestramento usiamo NAdam come algoritmo di ottimizzazione e il Mean Squared Error (MSE) come funzione di costo:

```
num_epochs = 120
loss_function = nn.MSELoss()
optimizer = torch.optim.NAdam([p for p in model.parameters() if p.requires_grad], lr=1e-3)
lr_scheduler = torch.optim.lr_scheduler.StepLR(
    optimizer,
    step_size=30, # every 30 epochs
    gamma=0.1     # reduce by factor of 10
)
```

Inoltre, definiamo uno scheduler del learning rate, che ha il compito di ridurre il learning rate di un fattore 10 ogni 30 epoche. Per comodità, definiamo una funzione per l'addestramento di un'epoca:

```
def train_one_epoch(model, loader, degradation, loss_function, optimizer, device):
    model.train() # abilita comportamenti specifici del training (es. dropout)
    cum_loss = 0.0
    num_images = 0
    pbar = tqdm.tqdm(loader)
    for images in pbar:
        images = images.to(device)
        noisy = degradation(images)

        optimizer.zero_grad() # (1) azzera i gradienti
        output = noisy - model(noisy) # (2) applica la rete con il residuo
        loss = loss_function(output, images) # (3) calcolo del valore di loss
        loss.backward() # (4) calcolo dei gradienti
        optimizer.step() # (5) aggiornamento dei pesi
        # Aggiornamento delle statistiche
        cum_loss += loss.item() * images.shape[0]
        num_images += images.shape[0]
    train_loss = cum_loss / num_images
    pbar.set_description(f'Loss: {train_loss:.4f}')
    return train_loss
```

E un'altra funzione per la valutazione, in cui, come indice prestazionale, utilizziamo il PSNR (Peak Signal-to-Noise Ratio):

```
def evaluate(model, loader, degradation, loss_function, device):
    model.eval() # disabilita comportamenti specifici del training
    epoch_cum_loss = 0.0
    epoch_cum_psnr = 0.0
    epoch_num_images = 0
    with torch.no_grad(): # disabilita il calcolo dei gradienti
        for images in tqdm.tqdm(loader):
            images = images.to(device)
            noisy = degradation(images)

            output = noisy - model(noisy)
            loss = loss_function(output, images)

            num_img = images.shape[0]
            epoch_cum_loss += loss.item() * num_img
            epoch_cum_psnr += torch.sum(-10*torch.log10(torch.mean((output- images)**2, (1,2,3))))
            epoch_num_images += num_img
    epoch_loss = epoch_cum_loss / epoch_num_images
    epoch_psnr = epoch_cum_psnr / epoch_num_images
    return epoch_loss, epoch_psnr
```

Eseguiamo il training per le varie epoche:

```
for epoch in range(num_epochs):
    # ---- Fase di TRAINING ----
    train_loss = train_one_epoch(model, train_loader, add_noise, loss_function, optimizer, device)
    print(f'Epoca [{epoch}/{num_epochs}] Loss train: {train_loss:.4f}')

    # ---- Fase di VALIDAZIONE ----
    val_loss, val_psnr = evaluate(model, val_loader, add_noise, loss_function, device)
    print(f'Epoca [{epoch}/{num_epochs}] Loss val: {val_loss:.4f} PSNR val: {val_psnr:.4f}')

    lr_scheduler.step() # conteggio dello scheduler del learning rate
```

Ricordate, a ogni epoca, di far avanzare lo scheduler del learning rate mediante la funzione `step()`.

1.4 Analisi delle prestazioni

Per evitare l'attesa dell'addestramento, possiamo scaricare i pesi della rete già addestrata nel modo seguente:

```
!wget -c -q https://www.grip.unina.it/download/guide_TF/dncnn_s50.pt
model.load_state_dict(torch.load('dncnn_s50.pt', map_location=device))
```

Valutiamo le prestazioni del modello sul test set:

```
test_loss, test_psnr = evaluate(model, test_loader, degradation_fun, loss_function, device)
print(f'Loss test: {test_loss:.4f} PSNR test: {test_psnr:.4f}')
```

Visualizziamo ora il risultato su un'immagine.

```
# Scarica immagine di test
!wget -q -c https://webpages.tuni.fi/foi/GCF-BM3D/images/image_Lena512rgb.png
```

```
import skimage.io as io
from torchvision.transforms.functional import to_tensor
img = to_tensor(io.imread('image_Lena512rgb.png')) # torch image
img = img.unsqueeze(0) # aggiungiamo la dimensione batch
noisy = add_noise(img) # degradazione

model.eval()
with torch.no_grad():
    noisy = noisy.to(device)
    result = noisy - model(noisy)

img = img.cpu().numpy()[0].transpose((1,2,0))
noisy = noisy.cpu().numpy()[0].transpose((1,2,0))
result = result.cpu().numpy()[0].transpose((1,2,0))

print('noisy PSNR:', -10*np.log10(np.mean((noisy-img)**2)))
print('result PSNR:', -10*np.log10(np.mean((result-img)**2)))

plt.figure()
plt.subplot(1, 3, 1); plt.imshow(noisy); plt.title('noisy')
plt.subplot(1, 3, 2); plt.imshow(result); plt.title('result')
plt.subplot(1, 3, 3); plt.imshow(img); plt.title('ground truth')
plt.show()
```

2 Restoration

Il codice per il denoising può essere adattato ad altri problemi di restoration semplicemente cambiando la *degradation function*. Provate a farlo per:

- il *deblurring*: considerando come degradazione uno smoothing gaussiano con deviazione standard pari a 7.
- la *super-resolution*: considerando come degradazione un rimpicciolimento dell'immagine riducendo righe e colonne di 4 volte, ricordate però di ripristinare le dimensioni iniziali mediante interpolazione lineare.